



Shifa Tameer-e-Millat University

شفّا تعميرِ ملّت يونيورسٹی

Software Design Specification (SDS DOCUMENT)

for

Professor OS

Version 1.1

By

Muhammad Yasin Asif BSSE-23S-0036

Muhammad Fahim BSSE-23S-0033

Rayyan Shahid BSSE-23S-0037

Supervisor

Ms. Habib-un-Nisa

Co-Supervisor

Mr. Muhammad Shaban Qabil

Bachelor of Science in Software Engineering (2023-2027)

Department of Software Engineering

Revision History

Name	Date	Reason for changes	Version
SDS Document	May,2026	Initial SDS submission all modules, FRs, NFRs, RTM complete	1.0
SDS Document	June,2026	Standardized identifier formatting; integrated React frontend visualization library (SI-8); decomposed consolidated Use Case diagrams into actor-specific flows for improved clarity.	1.1

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by

Ms. Habib-un-Nisa

Co- Supervised by

Mr.Muhammad Shaban Qabil

Table of Contents

Contents

1. Introduction.....	1
1.1 Modules Covered.....	1
2. Design Methodology and Software Process Model.....	2
2.1 Design Methodology.....	2
2.2 Software Process Model	2
3. System Overview	3
3.1 Architectural Design	3
4. Design Models	5
4.1 Class Diagram.....	6
4.2 Activity Diagrams.....	7
4.2.1 Activity Diagram — AI Grading Pipeline (M-03)	7
4.2.2 Activity Diagram — Student Submission and Feedback (M-05).....	9
4.3 Sequence Diagram	10
4.4 State Machine Diagram — Smart Proctoring (M-07)	11
4.5 Design and Implementation Constraints.....	12
5. Data Design.....	13
5.1 Data Dictionary	13
5.1.1 users	14
5.1.2 courses and assignments	15

5.1.3 rubrics and criteria	16
5.1.4 submissions	17
5.1.5 grading_results and criterion_results	18
5.1.6 explanations	19
5.1.7 proctor_sessions and focus_violations.....	20
5.1.8 chat_sessions and chat_messages	21
5.1.9 course_materials	22
6. Human Interface Design	22
6.1 Design Principles	22
6.2 System GUI.....	23
6.3 Integrated Code Editor and Compiler	24
6.5 Screen Objects and Actions	26
7. Algorithm Design.....	28
7.1 AI Grading and Evaluation (M-03).....	28
7.2 Explainable AI Processing (M-04)	29
7.3 Smart Proctoring (M-07).....	30
7.4 Code Editor and Compiler — Judge0 Execution (M-08)	31
7.5 AI Teaching Assistant — RAG Pipeline (M-09).....	33
8. References.....	35

1. Introduction

The system covers the whole assessment cycle: creating courses, building rubrics, AI-grading (text, code, MCQs), explaining those grades with LIME/SHAP, lightweight mobile proctoring (no cameras, just app-switch detection), a code editor with Judge0 compilation, analytics dashboards, and a 24/7 AI teaching assistant that answers questions using only the professor's uploaded materials.

The SRS already defined nine modules (M-01 to M-09) and their functional requirements. Now this SDS documents *how* we plan to build it: the architecture, the class diagrams, sequence flows, state machines, database design, and the actual constraints that shape our coding decisions.

1.1 Modules Covered

Module ID	Module Name	Platform	Responsible Team Member
M-01	Authentication and User Management	Web and Mobile	Rayyan Shahid
M-02	Course and Assignment Management	Web (Professor)	Rayyan Shahid
M-03	AI Grading and Evaluation	Backend	Muhammad Yasin Asif
M-04	Explainable AI (XAI) Processing	Web	Muhammad Yasin Asif
M-05	Student Submission Portal	Web (Student)	Rayyan Shahid
M-06	Analytics Dashboard	Web (Professor)	Muhammad Fahim
M-07	Smart Proctoring	Mobile (Flutter)	Muhammad Fahim
M-08	Code Editor and Compiler	Web and Mobile	Muhammad Fahim

M-09	AI Teaching Assistant	Web and Mobile	Muhammad Yasin Asif
------	-----------------------	----------------	---------------------

2. Design Methodology and Software Process Model

2.1 Design Methodology

We use object-oriented design because the domain naturally maps to objects. A course has Assignments each Assignment has a Rubric made of RubricCriteria, a Student submits a Submission, and an AIGradeResult attaches to that submission with explainable scores. Encapsulation keeps grading logic inside service classes. Inheritance lets us treat Professor, Student, TA, and Admin as specialised User types. Polymorphism allows different grading pipelines (text vs code vs MCQ) through the same interface. Backend (Python) and web frontend (TypeScript) both support OO patterns; Flutter uses a class-based component model.

2.2 Software Process Model

The team follows the Incremental Development model, organised into nine build phases. Each phase delivers a deployable and testable increment aligned to STMU FYP milestones.

Phase	Modules	Name	Key Deliverables
1	None	Project Scaffolding	Docker Compose, FastAPI skeleton, PostgreSQL schema, GitHub Actions CI/CD pipeline
2	M-01	Authentication	JWT auth, RBAC, Argon2id hashing, bulk CSV import, biometric mobile login
3	M-02	Course and Assignment	Course CRUD, rubric builder with weight validation, assignment creation, TA delegation
4	M-03	AI Grading Core	Celery queue, Llama 3.1 8B rubric-anchored grading, MCQ pipeline, human override UI

5	M-04	XAI	LIME token highlights, SHAP criterion overlays, Redis cache
6	M-05	Student Portal	Multi-type submission, XAI feedback viewer, grade dispute workflow, SSE notifications
7	M-06	Analytics Dashboard	Grade histogram, criterion radar chart, at-risk student table, cohort trend chart
8	M-07, M-08	Proctoring and Code Editor	Flutter Focus Mode with WebSocket sync [12], Monaco Editor with Judge0 sandboxed execution [11, 4]
9	M-09	AI Teaching Assistant	LangChain RAG pipeline over FAISS vector store, 24/7 course-specific Q&A

3. System Overview

ProfessorOS is a multi-tier web and mobile platform deployed entirely on university intranet hardware. It automates the complete academic assessment lifecycle from assignment creation through AI grading, explainable feedback, proctored examination, and performance analytics, with no internet access or paid cloud API required at any stage.

The system handles four submission types: text essay, MCQ, code, and file upload. It serves four user roles: Professor, Student, Teaching Assistant, and Administrator. It operates across two client surfaces: the React 18 web portal and the Flutter 3.22 mobile application. All AI inference runs locally via Ollama on the university server.

3.1 Architectural Design

We use a **layered architecture** because it separates concerns cleanly. Each layer has a well-defined responsibility and communicates only with the layer directly below (or via asynchronous queues).

Layers (bottom to top):

1. **Infrastructure Layer** – PostgreSQL (persistence), Redis (cache & broker), FAISS (vector store for course materials), Ollama (LLM inference), Judge0 (code sandbox) [7, 8, 4].
2. **Data Access Layer** – SQLAlchemy ORM, Redis client, FAISS wrapper.
3. **Business Logic / Service Layer** – Celery tasks for AI grading [10], XAI, RAG. Also proctoring event handler and rubric scoring engine.
4. **API / Application Layer** – FastAPI REST endpoints (JWT-protected) [5], WebSocket server for live proctoring, Server-Sent Events (SSE) for grade release.
5. **Presentation Layer** – React web portals (Professor, Student, Admin) and Flutter mobile app.

Communication rules:

- Web clients ↔ FastAPI: HTTPS/REST (or WSS for proctoring).
- FastAPI ↔ Celery: Redis as message broker (task queues).
- Celery workers ↔ Ollama/Judge0/sentence-transformers: direct HTTP or Python library calls.
- All containers talk over a dedicated Docker network; only Nginx reverse proxy exposes ports 80/443.

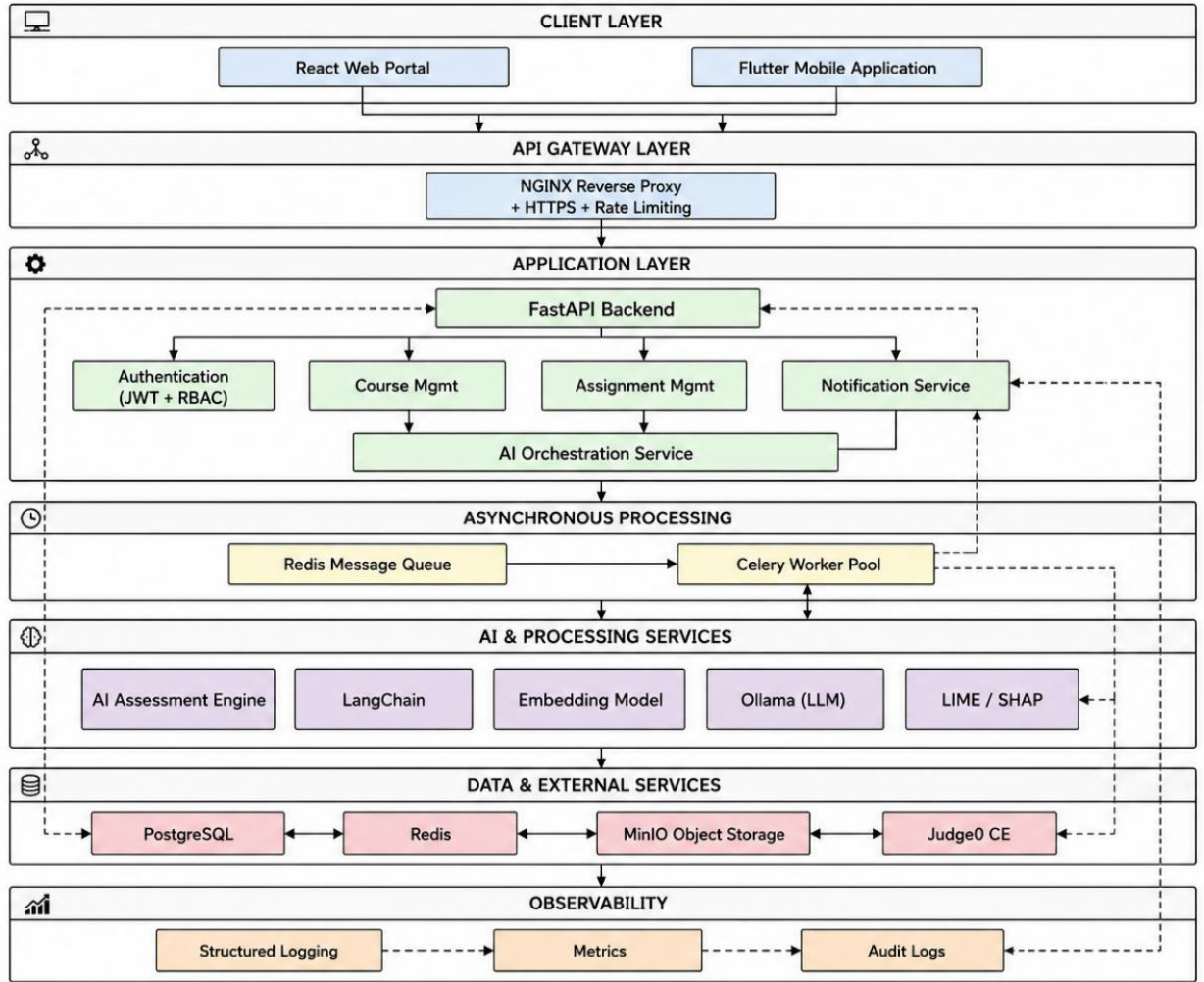


Figure 3.1 — Layered System Architecture Diagram

4. Design Models

ProfessorOS uses the Object-Oriented Development approach. The applicable UML design models are: Class Diagram, Activity Diagrams for major workflows, Sequence Diagram for the grading pipeline, and State Machine Diagram for Smart Proctoring. A design model syntax reference is provided in Appendix B.

4.1 Class Diagram

The class diagram shows all core domain classes, their attributes, methods, and inter-class relationships including generalization through inheritance, association, and composition.

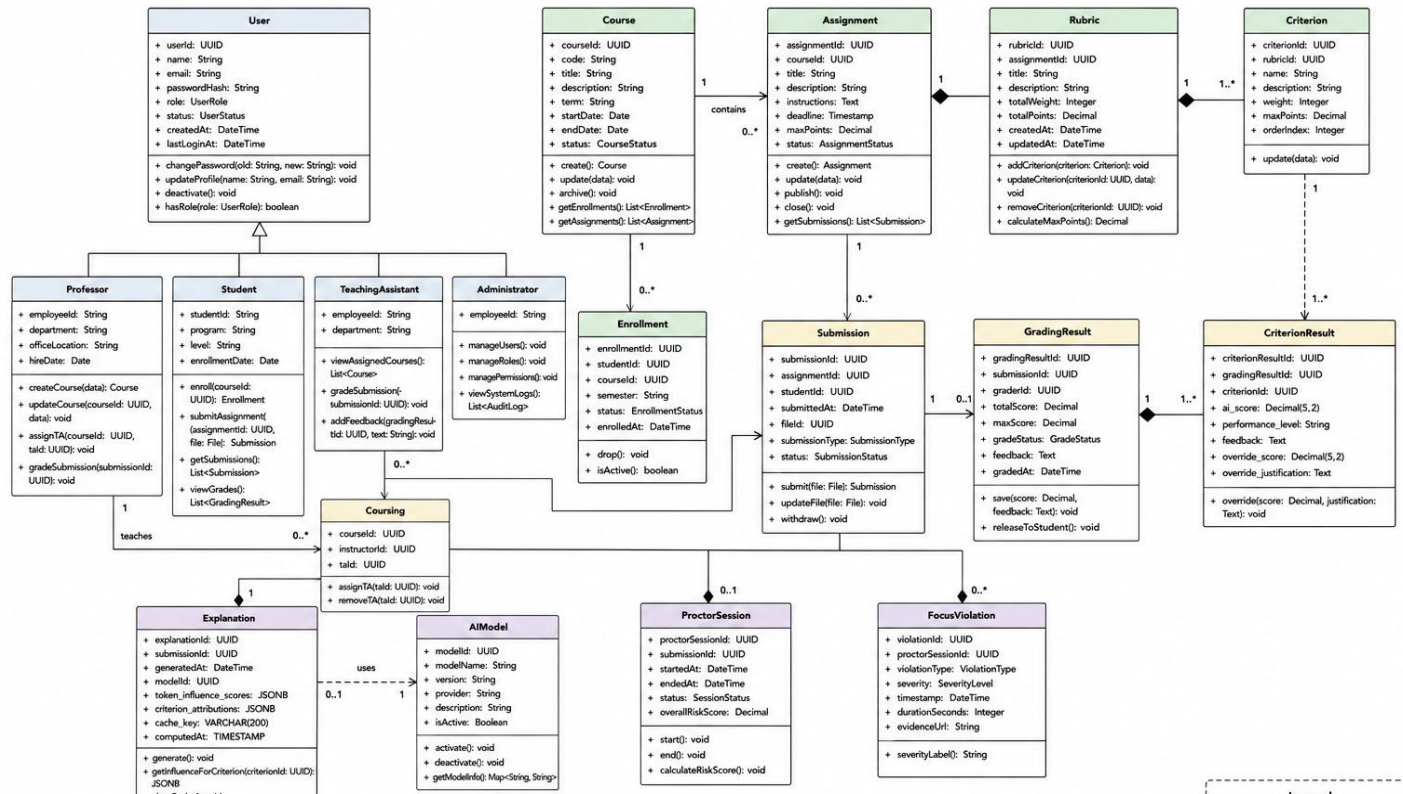


Figure 4.1 — Class Diagram — ProfessorOS Core Domain Model

1. User is abstract. Professor, Student, TeachingAssistant, and Administrator inherit from it, sharing base authentication attributes while adding role-specific ones.
2. GradingResult is composed of CriterionResult objects, one per rubric criterion. This lets the professor override a single criterion without touching others.
3. Explanation stores LIME token-level scores and SHAP criterion attributions as JSONB [1, 2], enabling sub-500ms Redis-cached serving on subsequent views.

4. ProctorSession aggregates FocusViolation objects independently logged and synced, supporting offline-first recording when the student device loses WiFi during the exam.

4.2 Activity Diagrams

Two activity diagrams are provided. The first covers the AI Grading Pipeline which is the most complex backend workflow. The second covers the Student Submission and Feedback workflow which is the highest-frequency user interaction path. Both use swimlanes to separate actor responsibilities.

4.2.1 Activity Diagram — AI Grading Pipeline (M-03)

This diagram uses four swimlanes: Student, FastAPI Backend, Celery Worker and Ollama AI, and Professor. It covers the full lifecycle from submission through asynchronous grading, XAI computation, professor review with optional override, grade approval, and student feedback delivery as shown in figure 4.2.1.

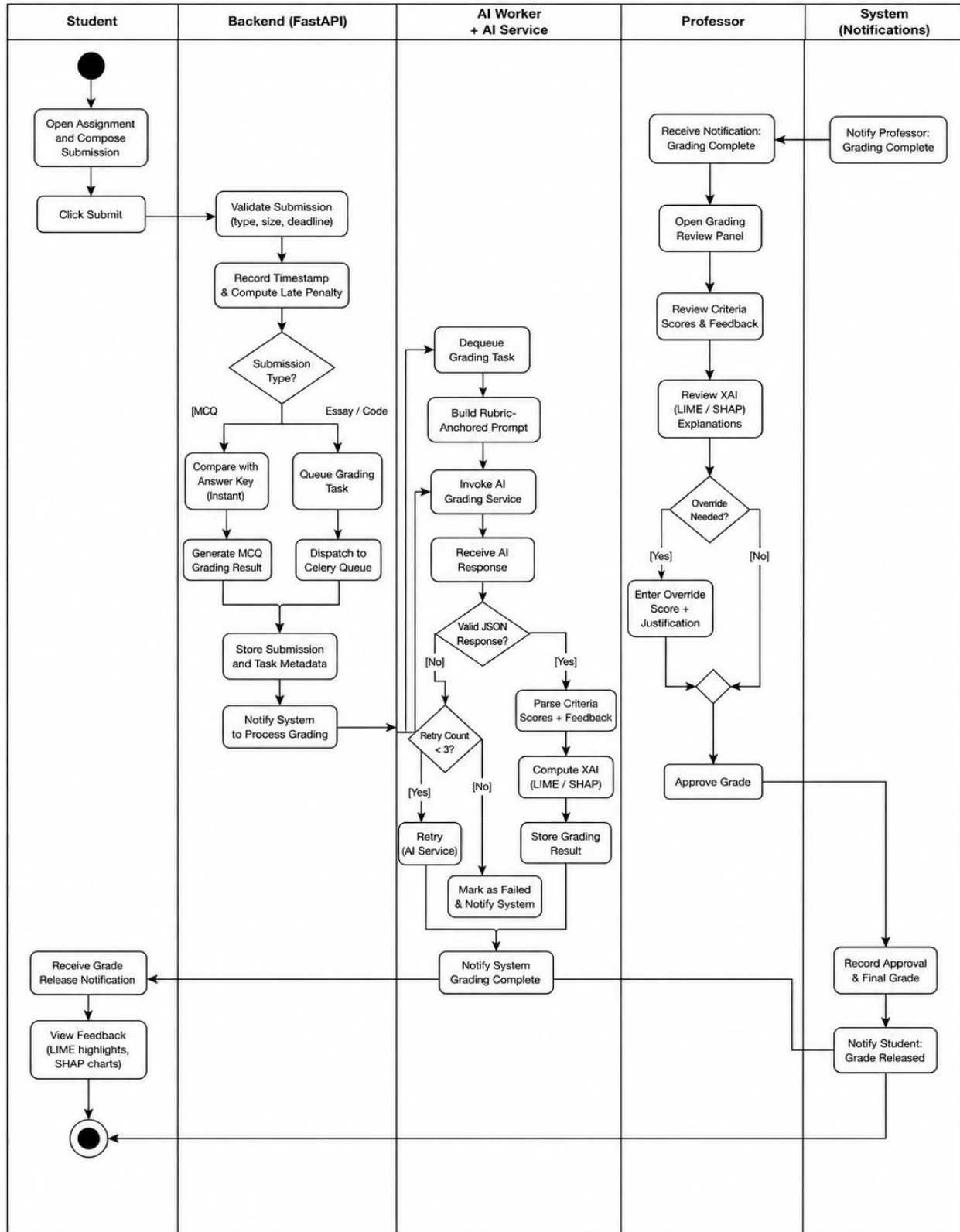


Figure 4.2.1 — Activity Diagram: AI Grading Pipeline (M-03) — ProfessorOS

4.2.2 Activity Diagram — Student Submission and Feedback (M-05)

This diagram covers the student-facing workflow using two swimlanes: Student and System. It shows selecting and composing a submission, submitting, waiting for AI grading, receiving the grade release notification, viewing XAI feedback, and optionally raising a grade dispute as shown in figure 4.2.2.

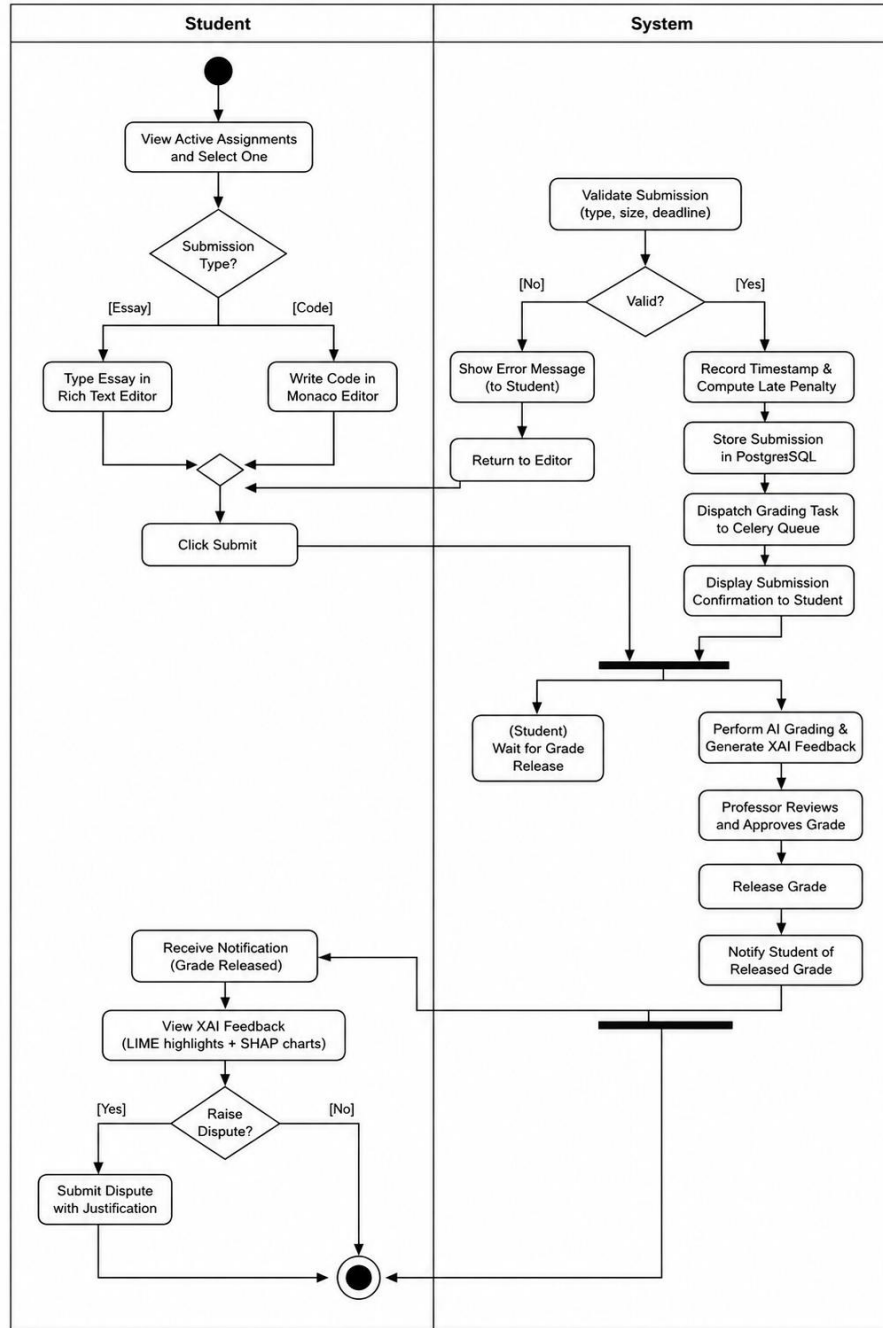


Figure 4.2.2 — Activity Diagram: Student Submission and Feedback

4.3 Sequence Diagram

The sequence diagram traces the complete interaction sequence from student submission through backend validation, Celery task dispatch, Ollama LLM inference, LIME and SHAP XAI computation, PostgreSQL storage, professor review, optional override, grade approval, and SSE notification to the student.

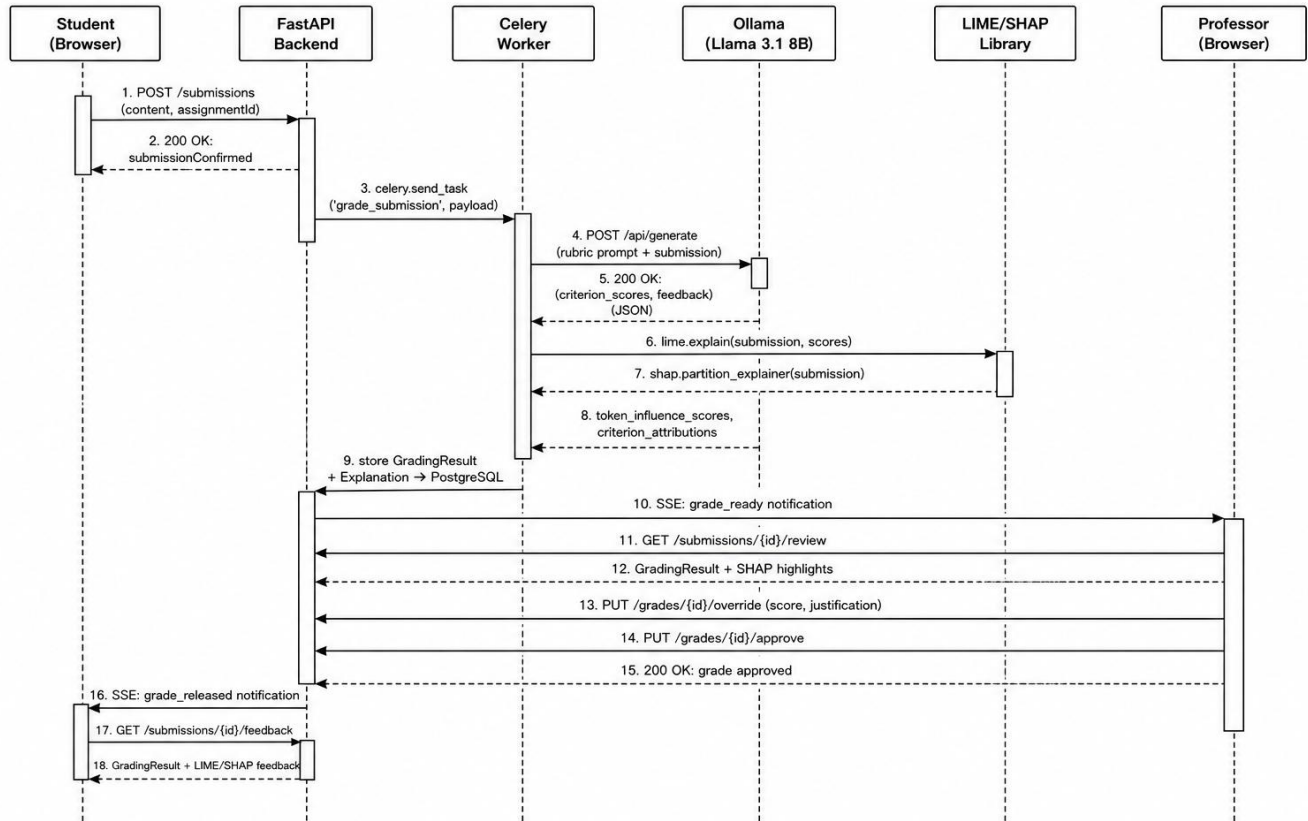


Figure 4.3 — Sequence Diagram: Grading Pipeline

4.4 State Machine Diagram — Smart Proctoring (M-07)

The state machine diagram models the complete lifecycle of a Smart Proctoring session on the Flutter mobile application. It covers all states: IDLE, FOCUS MODE ACTIVE, VIOLATION DETECTED, RETURNED IN TIME, CRITICAL VIOLATION, FLAGGED FOR REVIEW, DISCONNECTED, and SUBMITTED/COMPLETED.

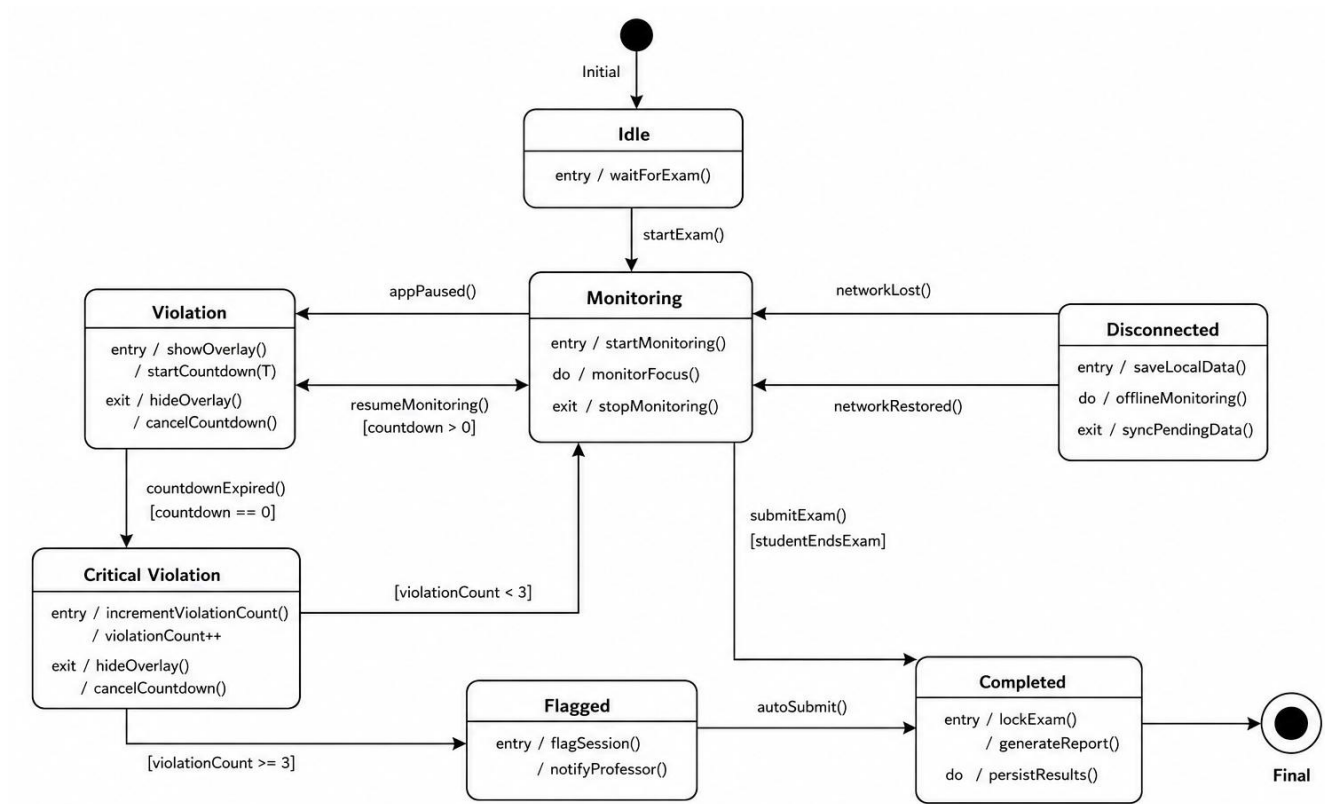


Figure 4.4 — State Machine Diagram: Smart Proctoring (M-07) — ProfessorOS

4.5 Design and Implementation Constraints

The following constraints apply to all development activities and cannot be overridden

ID	Constraint
CON-1	The system shall use Python 3.11 or later for all backend and AI/ML components. No Python 2.x code is permitted.
CON-2	LLM inference shall use Ollama running Llama 3.1 8B Instruct locally [3, 8]. No external OpenAI, Anthropic, or Gemini API calls are permitted in any production code path.
CON-3	All code execution must be routed through the self-hosted Judge0 CE sandbox. Student-submitted code must never execute on the FastAPI application server.
CON-4	All passwords must be hashed with Argon2id. MD5, SHA-1, SHA-256, and bcrypt are not acceptable alternatives.
CON-5	The React frontend must use Tailwind CSS 3.4 utility classes. Custom CSS overrides of Tailwind base classes are not permitted.
CON-6	The Flutter mobile app must use AppLifecycleState API for proctoring. Camera, microphone, or facial recognition APIs are explicitly forbidden.
CON-7	All database schema changes must be managed through Alembic migration scripts. Direct DDL modifications to the production database are not permitted.
CON-8	The entire system must be deployable via a single command: <code>docker compose up --build -d</code> . No manual server configuration steps are permitted outside of the <code>.env</code> file.
CON-9	All API endpoints must be documented via FastAPI auto-generated OpenAPI 3.0 at <code>/docs</code> and <code>/redoc</code> .

CON-10	Access tokens must not be stored in browser localStorage. They must be stored in httpOnly cookies or managed in-memory on the client.
--------	---

5. Data Design

We store persistent data in PostgreSQL. Transient data (cached explanations, session state) lives in Redis. Vector embeddings for course materials go into FAISS indexes one per course.

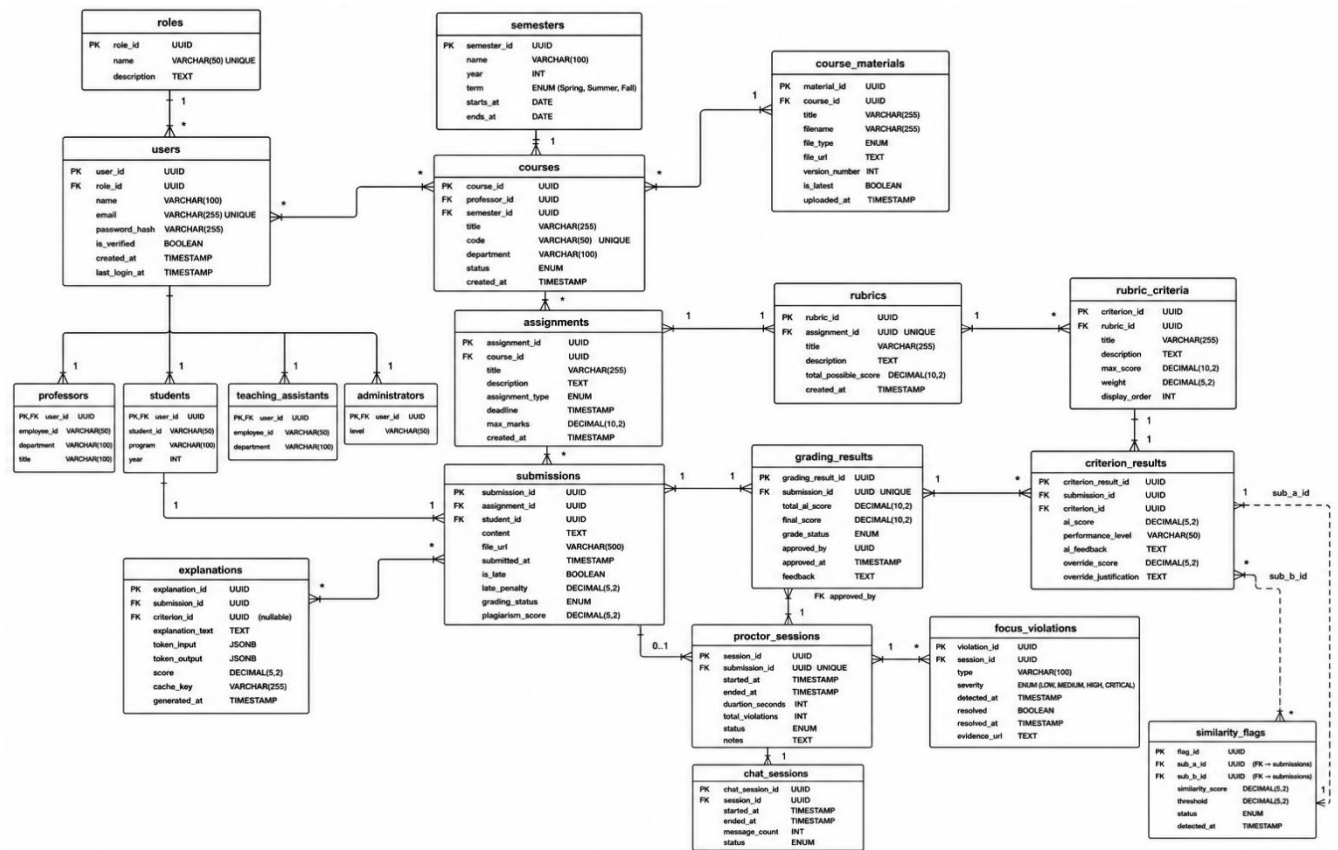


Figure 5.0 — Entity-Relationship / Data Model Diagram — ProfessorOS

5.1 Data Dictionary

The following data dictionary documents all major database entities. PK denotes Primary Key, FK denotes Foreign Key, and ENUM denotes a PostgreSQL enumerated type.

5.1.1 users

Attribute	Type	Constraint	Description
user_id	UUID	PK, NOT NULL	Unique identifier for each user account
full_name	VARCHAR(200)	NOT NULL	Full display name of the user
email	VARCHAR(255)	UNIQUE, NOT NULL	Institutional email used for login and verification
password_hash	VARCHAR(512)	NOT NULL	Argon2id hashed password. Plain text is never stored.
role	ENUM	NOT NULL	One of: PROFESSOR, STUDENT, TEACHING_ASSISTANT, ADMINISTRATOR
is_verified	BOOLEAN	DEFAULT FALSE	True once the user clicks the email verification link
is_active	BOOLEAN	DEFAULT TRUE	False when the account is suspended by an administrator
created_at	TIMESTAMP	NOT NULL	UTC timestamp of account creation

5.1.2 courses and assignments

Attribute	Type	Constraint	Description
course_id	UUID	PK, NOT NULL	Unique identifier for each course
professor_id	UUID	FK -> users	The professor who owns this course
title	VARCHAR	NOT NULL	Course display name
code	VARCHAR	NOT NULL	Institutional course code (e.g. CS-402)
semester	VARCHAR	NOT NULL	Active semester label (e.g. Fall 2026)
status	ENUM	NOT NULL	One of: ACTIVE, ARCHIVED, DELETED
assignment_id	UUID	PK, NOT NULL	Unique identifier for each assignment
course_id (FK)	UUID	FK -> courses	The course this assignment belongs to
submission_type	ENUM	NOT NULL	One of: TEXT_ESSAY, FILE_UPLOAD, MCQ, CODE
deadline	TIMESTAMP	NOT NULL	UTC cutoff for on-time submission
max_marks	INT	NOT NULL	Maximum achievable score for this assignment
status (assignment)	ENUM	NOT NULL	One of: DRAFT, PUBLISHED, CLOSED, ARCHIVED

5.1.3 rubrics and criteria

Attribute	Type	Constraint	Description
rubric_id	UUID	PK, NOT NULL	Unique identifier for the rubric
assignment_id	UUID	FK, UNIQUE	Exactly one rubric per assignment
total_weight	INT	DEFAULT 100	Sum of all criterion weights. Must equal 100 before publication.
criterion_id	UUID	PK, NOT NULL	Unique identifier for each rubric criterion
rubric_id (FK)	UUID	FK -> rubrics	The rubric this criterion belongs to
name	VARCHAR(200)	NOT NULL	Criterion label (e.g. Analytical Depth)
weight	INT	NOT NULL	Percentage weight of this criterion. All weights must sum to 100.
label	VARCHAR(50)	NOT NULL	One of: Excellent, Satisfactory, Developing, Insufficient
descriptor	TEXT	NOT NULL	Plain English description of what this level looks like

5.1.4 submissions

Attribute	Type	Constraint	Description
submission_id	UUID	PK, NOT NULL	Unique identifier for each student submission
assignment_id	UUID	FK -> assignments	The assignment this submission addresses
student_id	UUID	FK -> users	The student who submitted
content	TEXT	NULLABLE	Plain text content for essay and MCQ submissions
file_url	VARCHAR(512)	NULLABLE	Storage path for file upload submissions
submitted_at	TIMESTAMP	NOT NULL	Server-side UTC timestamp. Not modifiable by the student.
is_late	BOOLEAN	NOT NULL	True if submitted_at is after the assignment deadline
late_penalty_applied	DECIMAL(5, 2)	DEFAULT 0	Percentage penalty deducted from the final score
grading_status	ENUM	NOT NULL	One of: QUEUED, PROCESSING, COMPLETED, FAILED, APPROVED, FLAGGED

5.1.5 grading_results and criterion_results

Attribute	Type	Constraint	Description
result_id	UUID	PK, NOT NULL	Unique identifier for this grading result record
submission_id	UUID	FK, UNIQUE	One grading result per submission
total_score	DECIMAL(6, 2)	NOT NULL	Sum of all criterion scores. Reflects overrides if any were applied.
grade_status	ENUM	NOT NULL	One of: PENDING_REVIEW, APPROVED, FLAGGED, DISPUTED
approved_by	UUID	FK -> users, NULLABLE	Professor who approved this grading result
cr_id	UUID	PK, NOT NULL	Unique identifier for each criterion result row
criterion_id	UUID	FK -> criteria	The rubric criterion this result corresponds to
ai_score	DECIMAL(5, 2)	NOT NULL	Raw score assigned by the Llama 3.1 8B model
performance_level	VARCHAR(50)	NOT NULL	One of: Excellent, Satisfactory, Developing, Insufficient
ai_feedback_text	TEXT	NOT NULL	Natural language feedback generated by the LLM for this criterion
overridden_score	DECIMAL(5, 2)	NULLABLE	Score set by professor override. NULL if no override was performed.

override_justification	TEXT	NULLABLE	Mandatory justification text entered by the professor on override
------------------------	------	----------	---

5.1.6 explanations

Attribute	Type	Constraint	Description
exp_id	UUID	PK, NOT NULL	Unique identifier for this explanation record
submission_id	UUID	FK, UNIQUE	One explanation set per submission covering both LIME and SHAP
token_influence_scores	JSONB	NOT NULL	Array of {token, influence_score} objects produced by LIME
criterion_attributions	JSONB	NOT NULL	Array of {criterion_id, shapley_values[]} produced by SHAP
cache_key	VARCHAR(200)	UNIQUE	Redis key: explanation:{submission_id}:{type} with 30-day TTL
computed_at	TIMESTAMP	NOT NULL	UTC timestamp of initial LIME and SHAP computation

5.1.7 proctor_sessions and focus_violations

Attribute	Type	Constraint	Description
session_id	UUID	PK, NOT NULL	Unique identifier for the proctoring session
exam_id	UUID	FK -> assignments	The proctored examination assignment
student_id	UUID	FK -> users	The student being proctored
total_violations	INT	DEFAULT 0	Cumulative count of all focus violations detected in the session
flagged_for_review	BOOLEAN	DEFAULT FALSE	Set to TRUE when unresolved_violations reaches 3
violation_id	UUID	PK (focus_violations)	Unique identifier for each individual focus violation event
detected_at	TIMESTAMP	NOT NULL	UTC timestamp when AppLifecycleState transitioned to paused
resolved	BOOLEAN	DEFAULT FALSE	True if the student returned before the countdown timer expired
synced_to_server	BOOLEAN	DEFAULT FALSE	False while device is offline. Set to TRUE on successful WebSocket sync.

5.1.8 chat_sessions and chat_messages

Attribute	Type	Constraint	Description
session_id	UUID	PK, NOT NULL	Unique identifier for each AI Teaching Assistant chat session
student_id	UUID	FK -> users	The student who initiated the session
course_id	UUID	FK -> courses	The course context for RAG retrieval scope
msg_id	UUID	PK (chat_messages)	Unique identifier for each message in the session
role	ENUM	NOT NULL	One of: STUDENT, ASSISTANT (maps to LangChain message roles)
content	TEXT	NOT NULL	Message text content
source_docs	JSONB	NULLABLE	Array of source chunk references returned by RAG for assistant messages
timestamp	TIMESTAMP	NOT NULL	UTC timestamp of message creation

5.1.9 course_materials

Attribute	Type	Constraint	Description
material_id	UUID	PK, NOT NULL	Unique identifier for each uploaded course material
course_id	UUID	FK -> courses	The course this material belongs to
filename	VARCHAR(512)	NOT NULL	Original filename of the uploaded document
file_type	ENUM	NOT NULL	One of: PDF, DOCX, PPTX
chunk_count	INT	NOT NULL	Number of text chunks generated after splitting
uploaded_at	TIMESTAMP	NOT NULL	UTC timestamp of document upload and indexing

6. Human Interface Design

This section describes the user interface design principles applied across all ProfessorOS surfaces including the Professor Web Portal, the Student Web Portal, and the Flutter Mobile App. All interfaces use the Tailwind CSS 3.4 utility system and meet WCAG 2.1 Level AA contrast ratios with a minimum of 4.5:1 for normal text.

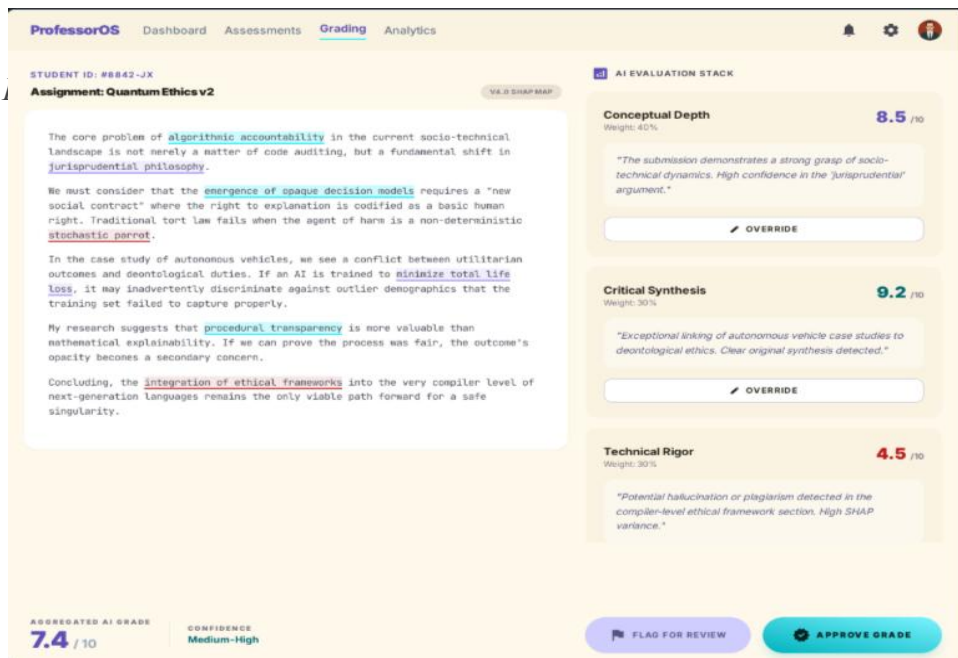
6.1 Design Principles

1. Minimum formatting: Interface chrome is kept clean with no sidebars and no popups for routine grading actions. Everything the professor needs during a review is visible on a single screen.

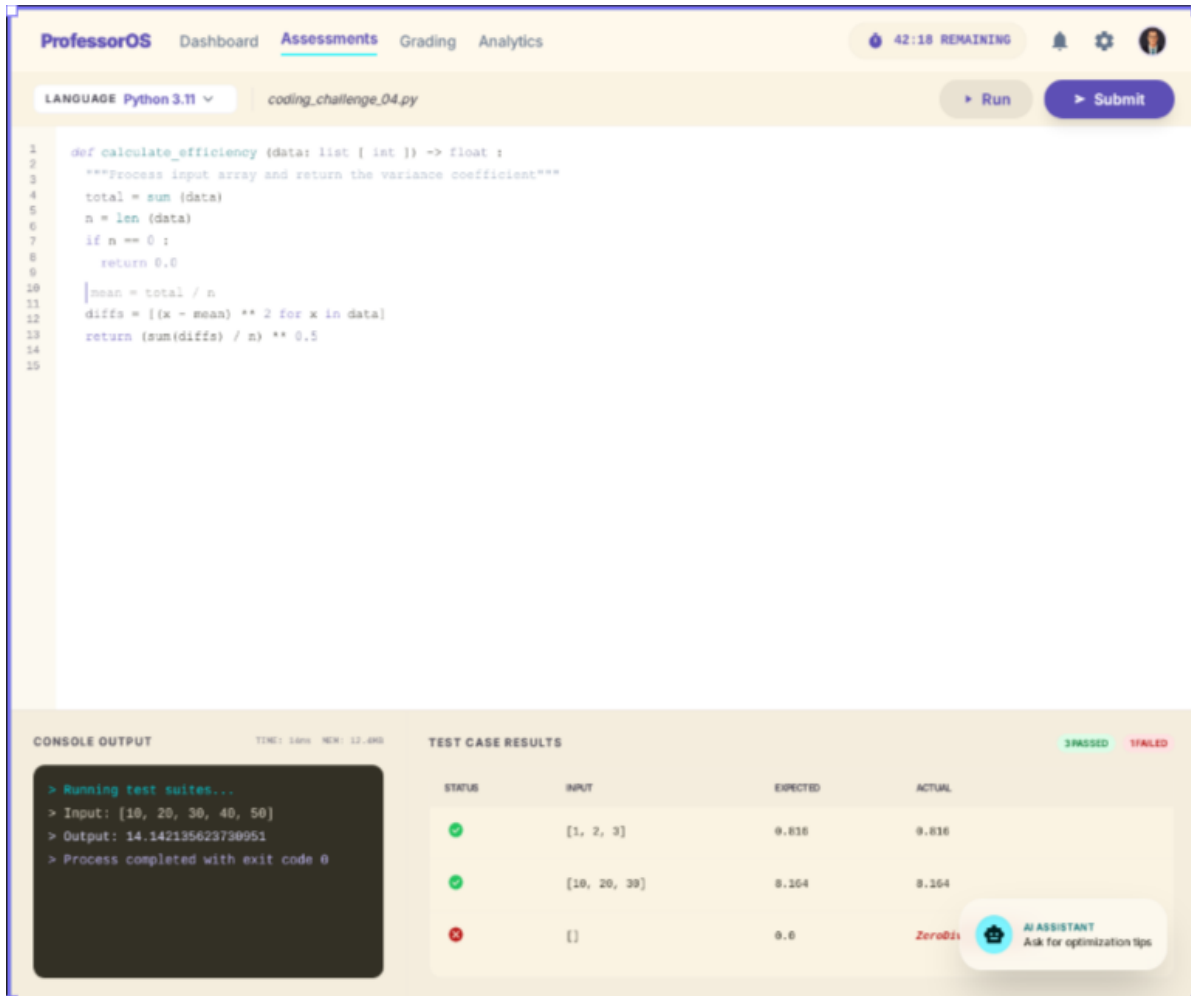
2. Semantic colour only: Green indicates positive contribution or approval, amber indicates neutral or developing status, and red indicates negative contribution or a flag. Colour is never used purely for decoration.
3. Permanent legend visibility: The SHAP colour scale legend is permanently visible on every feedback page without requiring the user to hover, expand, or navigate, as required by SRS USE-2.
4. Plain language: The student-facing feedback UI never exposes technical terms such as LIME, SHAP, or embedding, as required by SRS USE-3.
5. One-handed mobile operation: All exam interaction patterns on the Flutter app are reachable with the right thumb on screens of 5 inches or larger, as required by SRS USE-4.

6.2 System GUI

The GUI below shows the AI Grading Review Panel, the most complex and most frequently used professor-facing interface.



6.3 Integrated Code Editor and Compiler



The integrated code editor uses Monaco Editor on the web frontend and a native text input on the Flutter mobile app. Code execution is routed exclusively through the self-hosted Judge0 CE sandbox (CON-3). The system supports the following programming languages and their corresponding Judge0 language IDs:

Table 6.3.1 — Supported Programming Languages in Code Editor (M-08) — ProfessorOS

Language	Judge0 ID	Version
C	50	GCC 9.2.0
C++	54	G++ 9.2.0
Python 3	71	Python 3.8.1
Java	62	OpenJDK 13.0.1
JavaScript (Node.js)	63	Node.js 12.14.0

6.3 Exam Configuration and Scheduling

STEP 04 OF 05

Exam Configuration

Define logistics, proctoring security, and availability window.

Availability & Timeline

Set the window when students can start the assessment.

OPEN DATE

Oct 24, 2026
Saturday

CLOSE DATE

Oct 25, 2026
Sunday

09:00 AM
PST (UTC-8)

11:59 PM
PST (UTC-8)

VISUAL WINDOW PREVIEW

00:00 FRI EXAM WINDOW: 39 HOURS 00:00 SAT 00:00 SUN

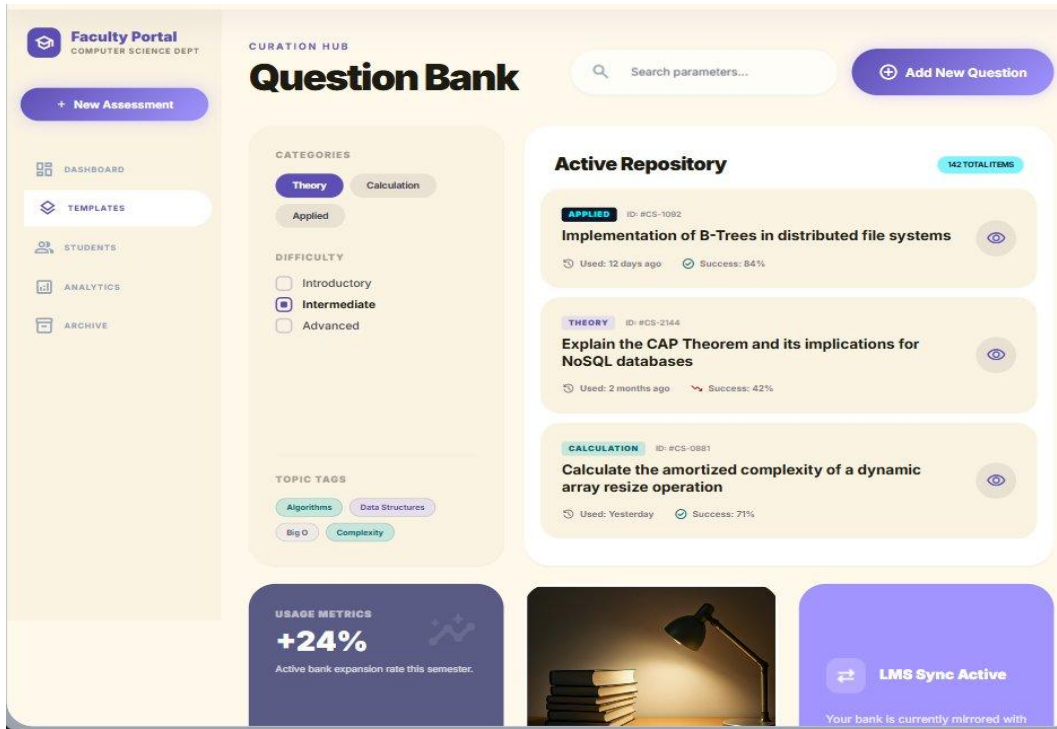
Duration

Total time permitted once the student starts.

CONFIGURATION CHECK

Back Review Exam

6.4 Question Bank and Creation Hub



6.5 Screen Objects and Actions

Screen Object	Type	Action / Description
SHAP Highlight in Green	Visual overlay on text	Non-interactive. Shows positive token contribution to criterion score. Intensity is proportional to Shapley value magnitude.
SHAP Highlight in Red	Visual overlay on text	Non-interactive. Shows negative token contribution. Hovering shows the exact token and Shapley value in a tooltip.
SHAP Colour Scale Legend	Static legend	Permanently visible below the submission text. No interaction. Educates the user without any navigation.

Criterion Card	Expandable card	Click to expand or collapse the SHAP bar chart showing the top 5 most influential tokens for that criterion.
AI Score Badge	Read-only badge	Displays the raw AI-assigned score. Replaced by the overridden score once the professor saves an override.
Performance Level Tag	Colour-coded tag	Green for Excellent, Blue for Satisfactory, Orange for Developing, Red for Insufficient.
Override Button	Primary action button	Opens an inline override input pre-filled with the AI score. Professor enters a new score and mandatory justification. Triggers FR-3.5 and creates an audit log entry.
AI Total Score Bar	Read-only summary	Displays the aggregated total score across all criteria. Updates automatically when any criterion override is saved.
Flag for Review	Warning action button	Sets grade_status to FLAGGED. Grade is not released to the student. Submission moves to the flagged review queue.
Approve Grade	Primary success button	Sets grade_status to APPROVED. Triggers an SSE notification to the student. Logs the approval event in the tamper-evident audit trail.
Navigation Bar	Fixed top bar	Contains Dashboard, Courses, Grading Queue, Analytics, and Settings. Accessible from all pages in the Professor Portal.

7. Algorithm Design

This section presents the pseudocode algorithms for five core modules of ProfessorOS. Each algorithm describes the key processing steps at a level of abstraction suitable for implementation guidance, without reproducing production code.

7.1 AI Grading and Evaluation (M-03)

The following algorithm describes the rubric-anchored LLM grading pipeline. MCQ submissions follow a direct comparison branch; text and file submissions are graded by Llama 3.1 8B via Ollama.

```
ALGORITHM: RubricAnchoredGrading(submission_id)

INPUT: submission_id : UUID

OUTPUT: GradingResult record inserted into PostgreSQL

1. FETCH submission, assignment, rubric, criteria[] FROM PostgreSQL

2. IF submission.grading_status != QUEUED THEN RETURN // already processed

3. SET submission.grading_status = PROCESSING

4. IF submission.type == MCQ THEN

    FOR EACH answer IN submission.answers:

        IF answer == correct_answer: correct_count += 1

    score = (correct_count / total_questions) * assignment.max_marks

    INSERT GradingResult(total_score=score, status=PENDING_REVIEW)

    RETURN

5. BUILD prompt:

    system: 'You are a university grading assistant...'

    rubric_context: criteria names, weights, performance descriptors

    submission_text: submission.content

    instruction: 'Return JSON: [{criterion_id, score, level, feedback}]'

6. CALL Ollama API: POST /api/generate

    { model: 'llama3.1:8b', prompt: prompt, stream: false }

7. PARSE JSON response into criterion_results[]
```

```
8. VALIDATE: SUM(criterion.score * criterion.weight / 100) == total_score
9. INSERT GradingResult(total_score, grade_status=PENDING_REVIEW)
10. FOR EACH cr IN criterion_results[:
    INSERT CriterionResult(criterion_id, ai_score, level, feedback)
11. UPDATE submission.grading_status = COMPLETED
12. DISPATCH xai_task.apply_async(args=[submission_id], countdown=2)
13. RETURN grading_result_id
```

7.2 Explainable AI Processing (M-04)

This algorithm computes LIME token-level influence scores and criterion attributions using criterion ablation [1, 2] (replacing KernelSHAP as described in CON-2 constraints). Results are Redis-cached for 30 days.

```
ALGORITHM: ExplainGrade(submission_id)
INPUT: submission_id : UUID
OUTPUT: Explanation record with token_influence_scores and criterion_attributions

1. FETCH submission.text, grading_result, criteria[] FROM PostgreSQL
2. CHECK Redis: key = 'explanation:{submission_id}'
   IF cache hit: RETURN cached explanation // sub-500ms serve

-- PART A: LIME Token Influence --
3. tokens[] = TOKENIZE(submission.text) // word-level split
4. baseline_score = grading_result.total_score
5. FOR EACH token t IN tokens[:
    masked_text = submission.text.replace(t, '[MASK]')
    masked_score = CALL_OLLAMA_SCORE(masked_text, rubric)
    influence[t] = baseline_score - masked_score
6. NORMALIZE influence[] to [-1.0, +1.0] using min-max scaling

-- PART B: Criterion Ablation (Shapley approximation) --
```

7. FOR EACH criterion *c* IN *criteria*[:
 contributions = []

 FOR EACH subset *S* of *criteria* NOT containing *c*:
 score_with_c = CALL_OLLAMA_SCORE(*text*, subset *S* + {*c*})
 score_without_c = CALL_OLLAMA_SCORE(*text*, subset *S*)
 contributions.append(*score_with_c* - *score_without_c*)

 shapley_value[*c*] = MEAN(*contributions*)
8. INSERT Explanation:
 token_influence_scores = [{*token*, *influence*}...] as JSONB
 criterion_attributions = [{*criterion_id*, *shapley_value*}...] as JSONB
9. CACHE in Redis: SET '*explanation:{submission_id}*' EX 2592000 // 30 days
10. RETURN *explanation_id*

7.3 Smart Proctoring (M-07)

The proctoring algorithm runs on the Flutter mobile application and uses the `AppLifecycleState` API. No camera, microphone, or biometric data is captured, as required by CON-6. Violations are buffered locally when offline and synced over WebSocket on reconnect.

ALGORITHM: `FocusMonitor(exam_id, student_id)`
INPUT: `exam_id, student_id` : UUID
OUTPUT: `ProctorSession` with `FocusViolation` records synced to server

1. CREATE `ProctorSession(exam_id, student_id)`
 SET `status = FOCUS_MODE_ACTIVE`
 SYNC to server via WebSocket
2. REGISTER listener: Flutter `AppLifecycleObserver`
3. LOOP WHILE `session.status != SUBMITTED`:
 event = AWAIT `next_lifecycle_event()`
 IF event == `AppLifecycleState.paused`:
 violation = CREATE `FocusViolation(detected_at=UTC_NOW())`

```
INCREMENT session.total_violations

SET session.status = VIOLATION_DETECTED

START countdown_timer(30 seconds)

IF session.total_violations >= 3:

    SET session.flagged_for_review = TRUE

    PUSH WebSocket event: { type: FLAG, session_id }

IF network_available:

    SYNC violation via WebSocket (synced_to_server = TRUE)

ELSE:

    BUFFER violation in Flutter SecureStorage

    SET violation.synced_to_server = FALSE

next_event = AWAIT lifecycle_event OR timer_expiry()

IF next_event == AppLifecycleState.resumed BEFORE timer:

    SET violation.resolved = TRUE

    SET session.status = RETURNED_IN_TIME

ELSE: // timer expired

    SET session.status = CRITICAL_VIOLATION

    IF assignment.auto_submit_on_critical: SUBMIT exam

IF event == AppLifecycleState.resumed:

    SET session.status = FOCUS_MODE_ACTIVE

IF network_reconnect_event:

    BATCH_SYNC all unsynced violations from SecureStorage

4. SET session.status = SUBMITTED

5. SYNC final session state to server

6. RETURN session_id
```

7.4 Code Editor and Compiler — Judge0 Execution (M-08)

All student-submitted code is executed inside the self-hosted Judge0 CE sandbox [4]. The FastAPI backend acts as a broker, submitting code and polling for results. Code never executes on the FastAPI server itself (CON-3).

ALGORITHM: ExecuteCode(source_code, language_id, stdin)

INPUT: source_code : string, language_id : int, stdin : string

OUTPUT: { stdout, stderr, status_label, time_ms, memory_kb }

SUPPORTED_LANGUAGES = {

50: 'C (GCC 9.2.0)',

54: 'C++ (G++ 9.2.0)',

62: 'Java (OpenJDK 13.0.1)',

63: 'JavaScript / Node.js 12.14.0',

71: 'Python 3.8.1'

}

1. IF language_id NOT IN SUPPORTED_LANGUAGES:

 RETURN Error('Language not supported')

2. payload = {

 source_code: BASE64_ENCODE(source_code),

 language_id: language_id,

 stdin: BASE64_ENCODE(stdin),

 cpu_time_limit: 5, // seconds

 memory_limit: 128000 // KB

}

3. response = POST http://judge0-ce:2358/submissions?wait=false

 BODY: payload

 token = response.token

4. retries = 0

5. LOOP:

 SLEEP(1 second)

 result = GET http://judge0-ce:2358/submissions/{token}

 IF result.status.id IN [1, 2]: // Queued or Processing

 retries += 1

```
        IF retries >= 10: RETURN Error('Execution timeout')

        CONTINUE

    ELSE: BREAK

6. STATUS_MAP = {

    3: 'Accepted', 4: 'Wrong Answer', 5: 'Time Limit Exceeded',

    6: 'Compilation Error', 7-12: 'Runtime Error'

}

7. RETURN {

    stdout: BASE64_DECODE(result.stdout),

    stderr: BASE64_DECODE(result.stderr),

    status: STATUS_MAP[result.status.id],

    time_ms: result.time * 1000,

    memory_kb: result.memory

}
```

7.5 AI Teaching Assistant — RAG Pipeline (M-09)

The teaching assistant uses LangChain with a FAISS vector store and sentence-transformers embeddings [6, 7, 9]. One FAISS index exists per course. The LLM is scoped strictly to uploaded course materials; it does not answer questions from its general training data.

```
ALGORITHM: RAGQuery(student_question, course_id, session_id)

INPUT: student_question : string, course_id : UUID, session_id : UUID

OUTPUT: { response : string, source_docs : chunk_reference[] }
```

1. LOAD FAISS index from disk:

```
path = /faiss_indexes/{course_id}/index.faiss

IF index not found: RETURN Error('No materials uploaded for this course')
```
2. `question_embedding = sentence_transformer.encode(student_question)`
`// Model: all-MiniLM-L6-v2, output dim = 384 [9]`
3. `distances[], chunk_ids[] = faiss_index.search(question_embedding, k=5)`
4. IF `MIN(distances) > SIMILARITY_THRESHOLD (0.75)`:

RETURN 'I can only answer questions based on uploaded course materials.'

5. FETCH chunk_texts[] FROM PostgreSQL WHERE chunk_id IN chunk_ids[]

6. context = JOIN(chunk_texts, separator='\n---\n')

7. prompt = f"

You are a teaching assistant for this course.

Answer using ONLY the following course materials. Do not use outside knowledge.

Materials: {context}

Student question: {student_question}

"

8. chain = PromptTemplate | OllamaLLM(model='llama3.1:8b') | StrOutputParser()

9. response = chain.invoke({ 'prompt': prompt })

10. STORE ChatMessage(session_id, role=ASSISTANT,

content=response, source_docs=chunk_ids)

11. RETURN { response: response, source_docs: chunk_references }

8. References

- [1] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135-1144.
- [2] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. 31st Conf. Neural Information Processing Systems (NeurIPS), 2017, pp. 4765-4774.
- [3] Meta AI Research, "Llama 3 Model Card," 2024. [Online]. Available: <https://ai.meta.com/research/publications/meta-llama-3/>
- [4] T. Akos, "Judge0 CE: An Open-Source Online Code Execution System," GitHub, 2020. [Online]. Available: <https://github.com/judge0/judge0>
- [5] S. Ramirez, "FastAPI: Modern, Fast Web Framework for Building APIs with Python," 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- [6] H. Chase, "LangChain: Building Applications with LLMs through Composability," GitHub, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [7] J. Johnson, M. Douze, and H. Jegou, "Billion-Scale Similarity Search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535-547, Jul. 2021.
- [8] Ollama Contributors, "Ollama: Get Up and Running with Large Language Models Locally," GitHub, 2023. [Online]. Available: <https://github.com/ollama/ollama>
- [9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. 2019 Conf. Empirical Methods in Natural Language Processing, 2019, pp. 3982-3992.
- [10] Ask Solem and contributors, "Celery: Distributed Task Queue," GitHub, 2009. [Online]. Available: <https://github.com/celery/celery>
- [11] Noctua Software, "Monaco Editor," GitHub, 2016. [Online]. Available: <https://github.com/microsoft/monaco-editor>
- [12] Google, "Flutter: Build apps for any screen," 2018. [Online]. Available: <https://flutter.dev>